Concurrency Control & Distributed Database Design Checkpoint

Part 1 - Transaction Management (Conceptual)

1. Concurrency Issue Identification

The primary concurrency anomaly that can occur in this scenario is a **Lost Update**. This happens when two transactions concurrently read the balance of the exact same account, calculate a new balance based on that stale initial value, and then sequentially write back their changes. As a result, the last transaction to commit completely overwrites and erases the financial update made by the first one.

2. Proposed Locking Mechanism

An **Exclusive Lock (X Lock)** must be applied to the affected bank accounts as soon as a transaction initiates a write sequence (debit or credit). While a Shared Lock (S Lock) allows multiple transactions to read data simultaneously, it is insufficient here; allowing multiple transactions to read a balance with the intent to modify it directly triggers the lost update problem. An exclusive lock ensures strict serializability by isolating the modifying data completely.

3. Approach Choice: Pessimistic vs. Optimistic Locking

In a core banking system, **Pessimistic Locking** is the preferred strategy.
Justification: Banking applications require zero-tolerance data integrity and absolute predictability. In an optimistic scheme, if a conflict is detected at the commit phase, the transaction must abort and roll back completely. For popular accounts or high-throughput systems, frequent transaction aborts drastically degrade user experience and waste computational cycles. Pessimistic locking prevents conflicts before they can happen by forcing concurrent transactions to wait gracefully.

4. Transaction Operations Schedule

Time	Transaction 1 (T1)	Transaction 2 (T2)	Account A Balance (Initial: $100)	Schedule Status
t1	Requests Exclusive Lock (Account A)	—	$100 (Locked by T1)	**SAFE** Transactions are completely serialized. T2 safely waits for T1 to finish, avoiding any lost updates.
t2	Reads Balance ($100)	Requests Exclusive Lock (Account A)	$100 (T2 queueing...)	
t3	Adds $50 (Local: $150)	Waiting...	$100 (Unchanged in DB)	
t4	Writes Balance & Releases Lock	Waiting...	**$150** (Updated in DB)	
t5	—	Obtains Exclusive Lock (Account A)	$150 (Locked by T2)	
t6	—	Reads Balance ($150)	$150 (T2 reads current state)	
t7	—	Subtracts $20 (Local: $130)	$150 (Unchanged in DB)	
t8	—	Writes Balance & Releases Lock	**$130** (Final DB State)	

Part 2 - Distributed Database Planning (High-Level)

1. Horizontal Fragmentation Strategy

The `Customers` table will be horizontally fragmented using a **Geographic Predicate Expression** based on the branch location to maximize data locality:

- **Fragment_Tunis:** σ Branch_Location = 'Tunis' (Customers) → Allocated to Tunis Server
- **Fragment_Sousse:** σ Branch_Location = 'Sousse' (Customers) → Allocated to Sousse Server
- **Fragment_Sfax:** σ Branch_Location = 'Sfax' (Customers) → Allocated to Sfax Server

2. Vertical Separation Strategy

The security credentials can be vertically partitioned into an isolated table called `Customer_Credentials`.

- **Columns Separated:** `Customer_ID` (Primary Key), `Login_Username`, `Password_Hash`, `Two_Factor_Secret`.
- **Justification:** These attributes are small, highly confidential, and accessed intensively during authentication. Separating them from bulky biographical data (like physical addresses or profile details) improves I/O cache efficiency for the login pipeline and hardens system security boundaries.

3. Data Replication Strategy

- **Data to Replicate Globally:** Basic Customer Metadata (e.g., `Customer_ID`, Full Name, Contact Info, Account Status).
- **Justification:** If a customer from Sfax travels to Tunis to perform an emergency over-the-counter request, the Tunis branch must instantly verify their identity and active account status locally. Making a blocking cross-network query to another branch for a simple name verification causes unacceptable latency and reliance on inter-branch network uptime.
  Note: Real-time account balances and transaction histories are kept strictly local to prevent severe distributed synchronization bottlenecks.

4. Allocation Strategy for Transaction History

A **Static Allocation** scheme must be utilized for transaction logs.

- **Justification:** Bank transaction history logs represent an append-only, legally unalterable ledger. Once a monetary transfer is finalized at the Sousse branch, that transaction record is historically permanent and will not shift. Dynamic allocation would

introduce useless distributed overhead tracking data that has zero operational reason to move.